

ARENA CARD GAME

System Architecture & Complete Design Document

Document Class	Technical Architecture & System Design
Version	1.0 -- Production Release
Platform	VIBE Learning Platform
Module	Arena Card Game (Knowledge Clash)
Backend Stack	Node.js / TypeScript / InversifyJS / MongoDB
AI Engine	Google Gemini (gemini-3.6-flash) via @google/genai SDK
Real-time Layer	Server-Sent Events (SSE) -- SseManager singleton
Date Generated	30 July 2026, 00:17

CONFIDENTIAL: This document is strictly scoped to the Arena Card Game subsystem. All sections, data, and logic described herein pertain exclusively to the Knowledge Clash arena engine, its game mechanics, AI pipeline, economy system, and administrative monitoring layer.

TABLE OF CONTENTS

1. System Overview & Key Features	3
2. Core Game Mechanics & Rules	5
2.1. Base Scoring & Hand Combination System	5
2.2. Power Card System & PowerUpEngine	7
2.3. Battle Session State Machine	9
3. Economy & Player Engagement Engine	11
3.1. Milestone Tier Architecture	11
3.2. Credit & Ticket Economy	12
3.3. HP Regeneration & Inventory Milestones	13
4. Real-Time Course Progression & AI Question Pipeline	14
4.1. Progress Fetcher & Contextual Boundary System	14
4.2. Robust AI Generator & Prompt Architecture	15
4.3. Batch Generation & Cost Optimisation	16
4.4. Offline Cache & Fallback System	17
4.5. Question Style Diversity Engine	18
5. Admin Control Arena Monitoring System	19
5.1. Teacher Dashboard Data Model	19
5.2. Infinite Credits Toggle & SSE Broadcast	20
5.3. AI Pipeline Health Metrics	21

1. SYSTEM OVERVIEW & KEY FEATURES

1.1 High-Level Introduction

The **Arena Card Game** -- branded as *Knowledge Clash* -- is an embedded, real-time educational card-battle engine deeply integrated into the VIBE Learning Platform. It transforms passive course consumption into an active, competitive knowledge assessment where learners battle an adaptive AI opponent using concept cards drawn strictly from their own completed course transcripts. The system is not a standalone game; it is tightly coupled to the learner's live progress data and serves as both a retention mechanism and a mastery checkpoint.

The architecture spans a full-stack TypeScript/Node.js backend (InversifyJS IoC, MongoDB), a React/TSX frontend, and a Gemini LLM pipeline. All game sessions are persisted in MongoDB collections (**arena_battle_sessions**, **arena_user_cards**, **arena_decks**) and real-time admin notifications are delivered via a Server-Sent Events (SSE) layer.

1.2 System Architecture Diagram

LEARNER PROFILE Course Enrollment Progress % (Live) Completed Item IDs	==> Progress API (/arena/courses)	ELIGIBILITY ENGINE evaluateArenaEligibility() Milestone Tier Unlock
AI QUESTION PIPELINE Gemini LLM (Batch x5) SegmentContextProvider	<== Transcript Context Completed VIDEO Items	BATTLE STATE MACHINE Round 1-5 / Ext. 6-10 PowerUpEngine
ADMIN MONITOR ArenaMonitorService SSE Broadcast	<=> Real-Time Events SseManager Singleton	KNOWLEDGE DASHBOARD Mastery % * Domain Breakdown Revision Targets

1.3 Key Positives & Design Strengths

- * **Contextually Bounded Questions:** All AI-generated questions are strictly scoped to the student's completed video transcripts -- never future content -- creating a fair, relevant assessment aligned with actual learning.
- * **Addictive Gamification Loop:** The combo scoring system (Pair to Full House) combined with random Power-Up drops creates a positive reinforcement loop that drives repeated play while ensuring each session is educationally purposeful.
- * **Adaptive Difficulty:** The computer AI's probability of answering correctly is dynamically computed, preventing trivially easy or impossibly hard opponents.
- * **Infinite Scalability via Batch AI:** Generating 5 questions per LLM call and caching them means a 5-round game consumes only 1 API call -- an 80% cost reduction.
- * **Course Progression Incentive:** The milestone credit economy explicitly gates arena access to course completion thresholds (30%, 50%, 70%, 90%, 100%), creating a direct behavioral incentive to progress through course material.

- * **Resilient Offline Fallback:** A 45-second AI timeout with a 3-stage MongoDB fallback ensures the game never crashes due to API failures.
- * **Real-time Admin Control:** Admins can toggle infinite arena credits for a course in real time; all enrolled students receive the update instantly via SSE.
- * **Strategy Depth via Power-Ups:** Six distinct power-ups with unique interaction patterns (offensive, defensive, multiplying) add a strategic layer beyond pure knowledge recall, rewarding planning and anticipation.

2. CORE GAME MECHANICS & RULES

2.1 Base Scoring & Hand Combination System

Every battle is scored by the `submitAnswer()` method inside `BattleService.ts`. The player submits a set of concept card names; the engine evaluates them against the `correctConcepts` array on the active question, counts matches, and resolves a `comboName` and multiplier. The base win award is **+50 points** before multipliers. A loss deducts **-30 points**, but points can never go below zero (non-negative floor rule enforced in code).

Base Win/Loss Points

Outcome	Base Points	Notes
Win (no mistakes)	+50 pts	Applied before multiplier
Loss (any mistake)	-30 pts	Capped so <code>totalPoints >= 0</code>
Shield-Blocked Loss	0 pts	Shield absorbs the deduction

Hand Combination Multiplier Table

Hand Combination	Correct Cards	Multiplier	Score Formula	Max Score
Single Strike	1 correct	1.0x	$50 \times 1.0 = 50$ pts	50 pts
Pair	2 correct	1.5x	$50 \times 1.5 = 75$ pts	75 pts
Three of a Kind	3 correct	2.5x	$50 \times 2.5 = 125$ pts	125 pts
Flush	4 correct	3.0x	$50 \times 3.0 = 150$ pts	150 pts
Full House	5+ correct	4.0x	$50 \times 4.0 = 200$ pts	200 pts*

* With Quick Counter permanent 2x multiplier active, Full House yields 400 pts/turn.

Score Calculation Pipeline

- * Step 1: Count correct submitted cards -- derive `correctCount` and `hasMistake`.
- * Step 2: Apply queued Power-Ups via `PowerUpEngine.apply()` (Strategy Pattern) -- may alter `correctCount`, `hasMistake`, or opponent `scoreDelta`.
- * Step 3: Clear `activePowerUps[]` array (single-round duration constraint).
- * Step 4: Assign `basePoints` (+50 Win / -30 Loss / 0 Shield-blocked) and multiplier based on combo tier.
- * Step 5: Compute `pointsEarned = round(basePoints x multiplier)`.
- * Step 6: Apply `permanentMultiplier` if > 1.0 and `pointsEarned > 0`.
- * Step 7: Enforce non-negative floor: if $\text{totalPoints} + \text{pointsEarned} \leq 0$, `pointsEarned = -totalPoints`.
- * Step 8: Check HP milestone (every 250 pts earned triggers +10 HP event).
- * Step 9: Check Power-Up milestone (every 100 pts of total drops a random power-up if inventory < 3).
- * Step 10: Advance round counter or set `AWAITING_EXTENSION` / `COMPLETED` status.

Non-Negative Points Floor Rule

A critical fairness constraint: the total score can never go below zero. If a loss penalty would result in a negative balance, the deduction is capped at the current total. This prevents discouraging death-spiral situations while still making losses meaningful.

```
if (totalPoints + pointsEarned <= 0) --> pointsEarned = -totalPoints; totalPoints = 0;
```

Adaptive Computer Opponent Logic

Probability Range	Computer Behavior	Combo Result
$P < 0.10$ (10%)	Selects random distractor cards only	Loss (-30 pts)
$0.10 \leq P < 0.40$ (30%)	Selects 1 correct card only	Single Strike
$0.40 \leq P < 0.70$ (30%)	Selects correct + 1 distractor (mistake)	Combo Broken
$P \geq 0.70$ (30%)	Selects all correct cards (full sweep)	Flush / Full House

2.2 Power Card System & PowerUpEngine

The Power-Up system is implemented using the **Strategy Pattern** in **PowerUpEngine.ts**. Each power-up is a discrete class implementing the **PowerUpStrategy** interface with a single **apply(context)** method. This design makes the system fully extensible -- new power-ups require no modification to existing code.

Key Architectural Constraints

- * **Single-Round Duration:** Power-ups are consumed and cleared from `activePowerUps[]` immediately after the round's evaluation, regardless of effect. Exception: Quick Counter writes to the `BattleSession`'s `permanentMultiplier` field, which persists for the entire session.
- * **Maximum Inventory:** The player can hold at most 3 Power-Ups at any time. Drops are silently discarded when inventory is full.
- * **Drop Rate:** One random power-up is awarded for every 100-point milestone crossed (e.g., 100 pts, 200 pts, 300 pts...).
- * **PowerUpEngine Registry:** All 6 strategies are pre-registered in a static map. Unknown power-up IDs produce a warning log without crashing the game.

The 6 Core Power-Ups

[SHIELD] Shield	
Effect	Prevents all point deduction on the next loss. The player receives 0 points instead of the standard -30 penalty. Consumed on use.
Engine Logic	<code>ShieldStrategy.apply()</code> sets <code>shieldUsed = true</code> and <code>basePoints = 0</code> when <code>hasMistake === true</code> . Has no effect on winning rounds.

[WILDCARD] Wildcard	
Effect	Substitutes for any one correct concept, incrementing the player's <code>correctCount</code> by +1, potentially upgrading their combo tier (e.g., Pair to Three of a Kind).
Engine Logic	<code>WildcardStrategy.apply()</code> executes <code>context.player.correctCount += 1</code> . This runs before multiplier assignment, so the extra count is fully factored into the combo evaluation.

[QUICK COUNTER] Quick Counter	
Effect	If the player wins the round in which Quick Counter is played, a permanent session-wide 2x multiplier is applied. This multiplier compounds and persists for the entire battle session.
Engine Logic	<code>QuickCounterStrategy</code> checks <code>!hasMistake && correctCount > 0</code> then executes <code>context.battle.permanentMultiplier *= 2</code> . The <code>permanentMultiplier</code> field lives on the <code>BattleSession</code> MongoDB document, making it session-bound and persistent.

[THE JOKER] The Joker	
Effect	Automatically plays all correct answers from the current deck, guaranteeing the highest possible combo on any question. Removes human error for one turn.

Engine Logic	TheJokerStrategy reads all isCorrect: true cards from the deck, overwrites submittedCards with their names, sets correctCount = correctCards.length, and forces hasMistake = false.
--------------	---

[REVERSAL] Reversal

Effect	Cast on the opponent; inverts their round result. A winning computer turn becomes a loss, and vice versa. Deals unexpected score swings.
Engine Logic	ReversalStrategy negates context.opponent.scoreDelta (scoreDelta = -scoreDelta) and updates the opponent's comboName to 'Reversed to Win!' or 'Reversed to Loss!'

[BLOCKER] Blocker

Effect	Nullifies the opponent's entire score gain for the current round, setting their delta to 0 regardless of their combo result.
Engine Logic	BlockerStrategy sets context.opponent.scoreDelta = 0, multiplier = 0, and comboName = 'Blocked!'

2.3 Battle Session State Machine

Each game is encapsulated in a **BattleSession** document persisted to MongoDB. The session holds all mutable game state and transitions through well-defined statuses:

Status	Trigger Condition	Next Allowed Actions
ACTIVE	startBattle() creates session	generateQuestion, submitAnswer
AWAITING_EXTENSION	Round 5 completed (maxRounds = 5)	extendBattle or concludeBattle
ACTIVE (Extended)	extendBattle() -- maxRounds set to 10	generateQuestion, submitAnswer (rounds 6-10)
COMPLETED	Round 10 submitAnswer or concludeBattle()	Read-only -- no further actions

Key BattleSession Fields

totalPoints	Cumulative player score for the session (non-negative floor enforced).
computerScore	Cumulative AI opponent score.
permanentMultiplier	Session-bound multiplier (default 1.0; Quick Counter doubles it each activation).
consecutiveWins	Tracks consecutive wins; reset to 0 on any loss.
inventory[]	Current held Power-Ups (max 3 strings).
activePowerUps[]	Power-Ups queued for the current round (wiped after apply()).
cachedQuestions[]	Pre-generated question batch cache (LIFO stack -- pop on use).
hpMilestoneProgress	Accumulator for HP milestone; decrements by 250 on each trigger.
lastPowerCardMilestone	Last 100-pt threshold at which a power-up was awarded.
currentRound/maxRounds	Round counter and cap (5 base, extendable to 10).
isActive	Boolean gate; false = game over, no further state mutations.

3. ECONOMY & PLAYER ENGAGEMENT ENGINE

3.1 Milestone Tier Architecture

The Arena's access economy is built on the `MILESTONE_TIERS` constant defined in `ArenaService.ts`. Each tier represents a course completion threshold that, once reached, unlocks one arena play credit. Credits are single-use per tier -- they are consumed when the player completes a battle at that threshold level and cannot be re-earned for the same tier. This creates a direct, irreversible incentive: progress the course to unlock more arena plays.

Tier Level	Course Progress Required	Bait HP Allocation	Credits Granted
Level 1	30% Completed	4 HP	1 arena play credit
Level 2	50% Completed	8 HP	1 arena play credit
Level 3	70% Completed	12 HP	1 arena play credit
Level 4	90% Completed	16 HP	1 arena play credit
Level 5	100% Completed	20 HP	1 arena play credit

DESIGN INTENT: The credit system builds healthy addiction by making arena access feel earned. A student who has completed 70% of a course has 3 credits (tiers 1, 2, 3), incentivising them to reach 90% for a 4th play without requiring external push notifications or daily bonuses.

3.2 Credit & Ticket Economy Deep-Dive

evaluateArenaEligibility() -- The Credit Calculator

This pure function (no side effects) is the single source of truth for determining a player's arena access rights. It is called from both the frontend (`ArenaDashboard.tsx`) and backend (`BattleService.startBattle()`) to ensure perfect consistency.

Output Field	Type	Description
unlockedTiers	MilestoneTier[]	All tiers whose threshold <= currentProgress
playableTiers	MilestoneTier[]	Unlocked tiers NOT yet in completedMilestones
availableCredits	number	Count of playableTiers (directly gates access)
activeTier	MilestoneTier	The first playable tier (lowest unplayed level)
nextLockedTier	MilestoneTier	Next tier not yet unlocked (shows progress goal)
isFullyCompleted	boolean	True when all 5 milestone thresholds are played

startBattle() -- Entry Gate Logic (Ordered Evaluation)

* **Step 1 -- Infinite Mode Check:** If `course.infiniteArenaEnabled === true` (admin-toggled), all credit and progress checks are bypassed. Credits = 999.

- * **Step 2 -- Minimum Progress Gate:** Rejects entry if course progress < 30%. Returns HTTP 403 with descriptive message.
- * **Step 3 -- Available Credits Gate:** Calls evaluateArenaEligibility() to confirm availableCredits > 0. Rejects with HTTP 403 if credits exhausted.
- * **Step 4 -- Session Cleanup:** Any existing active battle for the user is forcibly concluded (isActive = false) before creating the new session.
- * **Step 5 -- Session Initialisation:** Creates a new BattleSession with: 5 maxRounds, 1.0 permanentMultiplier, empty inventory, round 1.

3.3 HP Regeneration & Inventory Milestone Events

Milestone Type	Trigger	Reward	Reset Behaviour
HP Regeneration	Every 250 points earned in a session	+10 HP event fired to Global HP System	hpMilestoneProgress resets by -250 per trigger; multi-milestone jumps are iterated
Power-Up Drop	Every 100-point cumulative threshold (100, 200, 300...)	1 random Power-Up added to inventory (if slots < 3)	lastPowerCardMilestoneAchieved tracks last granted tier to prevent skips

HP Integration Note: The Arena Card Game tracks HP milestone progress internally (hpMilestoneProgress field) but does NOT store the player's actual HP value. HP is a global platform concern. The game fires triggerHpEvent: true in the submit response and the frontend triggers the external HP update -- clean separation of concerns.

4. REAL-TIME COURSE PROGRESSION & AI QUESTION PIPELINE

4.1 Progress Fetcher & Contextual Boundary System

The question generation pipeline is fed by a **real-time progress resolver** built into **BattleService.generateQuestion()**. Its purpose is to construct a precise list of completed video topics and their transcript context -- serving as the **ONLY** material the AI is permitted to draw from.

Step-by-Step Progress Fetch Logic

- * **Fetch Enrollments:** Retrieve all enrollments for the user via EnrollmentRepository. Find the **ACTIVE** enrollment for the current courseId.
- * **Extract completedItemIds:** Query the 'progress' MongoDB collection with isCompleted: true to get all item IDs the student has completed. Both ObjectId and string forms are matched for compatibility.
- * **Filter VIDEO Items Only:** For each completed item ID, call itemRepo.readItemById() and keep only items where type === 'VIDEO'. This strictly excludes quiz metadata, PDFs, and structural modules.
- * **Fetch Transcripts:** For each selected VIDEO item, call segmentContextProvider.getContext({segmentId, courseVersionId}) to retrieve the full transcript text.
- * **Assemble Context String:** Concatenate all transcripts into a single structured context block with topic labels.

***STRICT SCOPING GUARANTEE:** The AI prompt contains an explicit hard boundary: 'YOU ARE STRICTLY RESTRICTED TO THESE COMPLETED TOPICS ONLY. DO NOT ASK QUESTIONS OR GENERATE CARDS FOR UNLEARNED/FUTURE CONCEPTS OUTSIDE THIS LIST.' This is enforced in the prompt text itself.*

4.2 Robust AI Generator & Prompt Architecture

The AI generation prompt is a meticulously structured system prompt sent to **Google Gemini** via the @google/genai SDK. It enforces four critical rules and a typed JSON output schema to guarantee predictable, game-ready question data.

The 4 Critical Rules Embedded in Every Prompt

Rule	Directive	Purpose
CRITICAL RULE 1	STRICT SCOPING BY PROGRESS: questions ONLY from completed topics at the student's current % boundary	Prevents unfair or confusing out-of-scope questions
CRITICAL RULE 2	QUESTION VARIETY: use the randomly selected style for this turn	Forces AI to avoid repetitive question formats
CRITICAL RULE 3	PUNCHY & CONCISE: full question readable in < 10 seconds; card names 1-4 words; explanations 1 sentence	Optimises for the fast-paced card game format

CRITICAL RULE 4	IGNORE COURSE STRUCTURE METADATA: no 'video segments', 'quizzes', 'modules' or 'transcripts' in questions	Ensures questions are about subject matter, not platform mechanics
-----------------	---	--

Structured JSON Response Schema

- * Root object { batch: ArenaQuestion[] } -- array of 5 questions.
- * Each question: { promptText: string, deck: Card[], explanation: string }.
- * Each card: { name: string, explanation: string, isCorrect: boolean } -- all three fields required.
- * Deck size: exactly 5 cards per question (mix of correct answers and distractors).
- * Schema enforced via Gemini's native responseMimeType: 'application/json' mode.

4.3 Batch Generation & Cost Optimisation

Instead of generating one question per API call (the naive approach), the system requests a batch of **5 questions per LLM call**. This is the primary mechanism for minimising both LLM API costs and MongoDB read/write latency.

Cache Architecture (LIFO Stack)

- * **On first question request:** Cache is empty -- LLM call made -- 5 questions returned -- stored in `cachedQuestions[]` -- 1 popped as `currentQuestion`.
- * **On subsequent requests (rounds 2-5):** Cache is non-empty -- `cachedQuestions.pop()` returns the next question instantly. Zero LLM API calls.
- * **After round 5 extension:** If the player extends to 10 rounds, a new LLM batch call is triggered for rounds 6-10 when the cache is exhausted.
- * **Database persists the cache:** Each pop and push results in a MongoDB document save -- server restarts cannot lose cached questions.

Cost Reduction Analysis

Game Length	Naive Approach (1 call/round)	Batch Approach	Cost Savings
5 rounds (base)	5 LLM API calls	1 LLM API call	80% reduction
10 rounds (extended)	10 LLM API calls	2 LLM API calls	80% reduction
Avg session (7 rounds)	7 LLM API calls	2 LLM API calls	~71% reduction

4.4 Offline Cache & Fallback System

The AI pipeline includes a 4-stage fallback system to guarantee game continuity even when the LLM API is unavailable, slow, or returns malformed output. Every Gemini API call is wrapped in a `Promise.race()` against a **45-second timeout promise**.

```
const timeoutPromise = new Promise((_, reject) => setTimeout(() => reject(new Error('LLM Request Timeout (>45s)'), 45000)); const response = await Promise.race([aiCall, timeoutPromise]);
```

4-Stage Fallback Chain

Stage	Trigger	Action	MongoDB Query
Stage 1 -- Primary	Normal flow	Generate 5 questions via Gemini LLM	None (external API)
Stage 2 -- Topic Fallback	LLM timeout (>45s) or parse error	Fetch cached question from MongoDB 'questions' collection matching <code>completedTopics[]</code>	<code>\$match { topic: { \$in: completedTopics } } + \$sample { size: 1 }</code>
Stage 3 -- Course Fallback	No topic-matching questions found	Fetch any question matching current <code>courseId</code>	<code>\$match { courseId } + \$sample { size: 1 }</code>
Stage 4 -- Last Resort	No course-matching questions found	Fetch completely random question from questions collection	<code>\$sample { size: 1 } (no filter)</code>

Topic Shuffling for Variety: When the fallback is triggered, completedTopics are passed in shuffled form (the main generator already shuffles topic selection via `[...videoItems].sort(() => 0.5 - Math.random())`), so different fallback calls match different cached questions -- providing variety even in offline mode.

4.5 Question Style Diversity Engine

To prevent the AI from generating repetitive question formats, the pipeline includes a **Question Style Selector** that randomly picks one of 6 distinct pedagogical styles for each batch generation request. The selected style is injected directly into CRITICAL RULE 2 of the prompt.

Style Name	Instruction to AI	Pedagogical Goal
Scenario Analysis	Create a practical problem; choose concepts required to solve it.	Applied problem-solving; assesses real-world application.
Fill in the Blank	Create a statement with a crucial concept missing; select the card that fills it.	Precise concept recall; tests definitional accuracy.
Concept Comparison	Contrast key concepts; select the true matching concepts.	Discrimination between similar ideas; prevents confusion.
Diagnostic Logic	Describe an error or bug; select cards with corrective concepts.	Debugging mindset; tests error recognition.
Practical Application	Formulate an execution task; select correct implementation steps/tools.	Procedure knowledge; tests how to do vs. what it is.
Principle Identification	Present a specific outcome; pick fundamental principles that govern it.	Conceptual depth; tests underlying principles.

5. ADMIN CONTROL ARENA MONITORING SYSTEM

5.1 Teacher Dashboard Data Model

The **ArenaMonitorService** provides the backend data layer for the admin dashboard. Its primary method, `getTeacherCoursesWithStats()`, aggregates real-time data from four MongoDB collections (`newCourse`, `user`, `enrollment`, and the arena repository) into a structured stats object per course.

Per-Course Statistics Object

courseId	MongoDB ObjectId string for the course.
courseName	Human-readable course title.
description	Course description for admin context.
infiniteArenaEnabled	Boolean: Whether admin has toggled unlimited play mode.
totalEnrolled	Count of ACTIVE enrollments for this course.
students[]	Array of per-student monitoring objects (see below).

Per-Student Monitoring Fields

Field	Source	Description
<code>userId</code>	<code>enrollment.userId</code>	Student's user ID
<code>name</code>	<code>user.name</code> / <code>displayName</code> / email prefix	Display name (multiple fallbacks)
<code>email</code>	<code>user.email</code>	Student contact email
<code>progressPercent</code>	<code>enrollment.percentCompleted</code>	Live course completion %
<code>completedMilestones</code>	<code>enrollment.arenaProgress.completedMilestones</code>	Array of threshold values played
<code>availableCredits</code>	<code>evaluateArenaEligibility()</code>	Remaining playable credits
<code>turnsPlayed</code>	<code>completedMilestones.length</code>	Number of arena sessions completed

5.2 Infinite Credits Toggle & SSE Real-Time Broadcast

The most powerful admin capability is the ability to **toggle infinite arena credits for a course in real time**. This allows teachers to run unlimited practice sessions for exam preparation, review weeks, or competitive events without modifying student data or course structure.

`toggleInfiniteCredits()` -- Operation Flow

* **Input:** `courseId` (string) + `enabled` (boolean).

- * **Atomic Update:** MongoDB updateOne() sets { infiniteArenaEnabled: enabled, updatedAt: new Date() } on the course document. Throws 404-equivalent error if no course matched.
- * **SSE Broadcast:** Immediately after the DB write, SseManager.getInstance().broadcast() pushes an INFINITE_CREDITS_TOGGLED event to all connected clients.
- * **Client Update:** The ArenaDashboard SSE listener on the frontend receives the event and updates the specific course's infiniteArenaEnabled flag in local React state. Zero page reloads required.

SseManager -- Singleton Architecture

Method	Signature	Description
getInstance()	static -> SseManager	Returns/creates the single global instance.
addClient(res)	Response -> void	Registers a new EventSource client stream.
removeClient(res)	Response -> void	Deregisters a disconnected client.
broadcast(event)	SseEvent -> void	Writes JSON event to all registered streams.
getClientCount()	-> number	Returns active connected client count.

Resilience: If a client write throws (e.g., connection dropped), the SseManager automatically removes that client from the set and continues broadcasting to remaining clients without interruption.

5.3 AI Pipeline Health & Economy Metrics

Metric	Data Source	Operational Significance
Avg. Credits Remaining per Student	evaluateArenaEligibility() per enrollment	Indicates whether students are progressing through content (low credits = high engagement)
Students at 0 Credits	availableCredits === 0 count	Identifies highly engaged students who need infinite mode to continue
Total Arena Sessions Played	Sum of completedMilestones.length across enrollments	Measures overall engagement with the Arena subsystem
Course Progress Distribution	Histogram of progressPercent values	Shows if credit gates are correctly positioned within the course flow
Infinite Arena Courses	infiniteArenaEnabled flag count	Shows how many courses are in unrestricted admin mode
SSE Connected Clients	SseManager.getClientCount()	Real-time measure of active admin dashboard watchers

Admin API Endpoint Summary

Endpoint	Method	Description
/arena/courses	GET	Student: fetch enrolled courses with eligibility data
/:courseId/battle/start	POST	Initiate a new battle session (credit check enforced)

/battle/:id/question	POST	Fetch next question (cache -> AI -> fallback chain)
/battle/:id/submit	POST	Submit answer + optional power-up; receive scored result
/battle/:id/extend	POST	Extend from 5 to 10 rounds (one-time per session)
/battle/:id/conclude	POST	Manually end battle early
/status/:courseId	GET	Player's current eligibility & milestone state
/complete-milestone	POST	Record a played milestone threshold (atomic)
/consume-credit	POST	Alias for recordTurn -- consume one arena credit
/events/stream	GET	SSE stream endpoint for real-time admin events

DOCUMENT SUMMARY & SOURCE REFERENCES

This document has provided a complete, code-verified technical specification of the **Arena Card Game** system as implemented within the VIBE Learning Platform. All sections have been synthesised directly from the following verified source files:

Source File	Module	Contents Documented
ArenaService.ts	Backend / services	MILESTONE_TIERS, evaluateArenaEligibility, credit economy, milestone completion
BattleService.ts	Backend / services	startBattle, generateQuestion, submitAnswer, extendBattle, 4-stage fallback
PowerUpEngine.ts	Backend / services	All 6 PowerUpStrategy implementations, Strategy Pattern architecture
ArenaMonitorService.ts	Backend / services	Teacher dashboard aggregation, toggleInfiniteCredits, SSE integration
SseManager.ts	Backend / services	Real-time SSE broadcast singleton, client lifecycle management
ArenaController.ts	Backend / controllers	All REST endpoints with @Authorized guards
BattleSession.ts	Backend / transformers	Complete BattleSession schema and field semantics
ArenaDashboard.tsx	Frontend / pages	SSE listener, milestone tier mirroring, eligibility display
Card_Game_Point.md	Documentation	Game rules, combination tables, HP regeneration spec
pvc_card_game_arch.md	Documentation	PvC architecture spec, AI dealer prompt structure

All documented features, rules, multipliers, thresholds, timeout values, API endpoints, and data structures reflect the actual implemented source code as of the document generation date. No placeholder, hallucinated, or out-of-scope content has been included in this specification.

Generated by VIBE Platform -- Antigravity Technical Writer | 30 July 2026 at 00:17